100 Exam-Style Practice Questions

Databricks Certified **Machine Learning Associate**

4 options per question. Answer key with explanations on the final pages.

Practice Exam

Contents

Instructions **2**

1 A. Databricks Machine Learning **3**

2 B. ML Workflows **9**

3 C. Model Development **12**

4 D. Model Deployment **17**

5 Answer Key with Explanations **19**

Instructions

Each question has exactly four options (A–D). Choose the single best answer. The answer key with a short explanation of why the correct option is right and why the other three are wrong begins on page 19.

Domains: A. Databricks Machine Learning (Q1–38); B. ML Workflows (Q39–57); C. Model Development (Q58–88); D. Model Deployment (Q89–100).

1 A. Databricks Machine Learning

Q1. A machine learning engineer created a Feature Table `new_table` using Feature Store Client `fs` and specified a metadata description. Which line of code returns that description?

(a) `fs.create_training_set("new_table")`

(b) `fs.get_table("new_table").description`

(c) `fs.get_table("new_table").load_df()`

(d) `fs.get_table("new_table")`

Q2. What is the primary advantage of creating Feature Store tables at the account level in Unity Catalog rather than at the workspace level?

(a) Faster model training

(b) Cross-workspace governance, discoverability, and access control

(c) Automatic feature engineering

(d) Lower storage cost

Q3. Which client is used to create and manage Feature Store tables in Unity Catalog?

(a) `FeatureStoreClient`

(b) `FeatureEngineeringClient`

(c) `UnityCatalogClient`

(d) `MLflowClient`

Q4. Which method creates a feature table with `FeatureEngineeringClient`?

(a) `fe.create_table()`

(b) `fe.write_table()`

(c) `fe.create_training_set()`

(d) `fe.log_model()`

Q5. Which method writes features to an existing feature table?

(a) `fe.create_table()`

(b) `fe.write_table()`

(c) `fe.score_batch()`

(d) `fe.get_table()`

Q6. What does `fe.create_training_set()` provide that a plain DataFrame join does not?

(a) Automatic one-hot encoding

(b) Point-in-time lookups aligned to the label timestamp

(c) Model registration

(d) Hyperparameter tuning

Q7. A model performs well offline but degrades in production. Review shows a feature was computed using data available only after the label outcome. What is the diagnosis?

(a) Underfitting

(b) Training-serving skew / target leakage

(c) Class imbalance

(d) Concept drift

Q8. What is the difference between online and offline feature tables?

(a) Online is for training; offline is for serving

(b) Online serves low-latency real-time inference; offline is for batch training and scoring

(c) They are identical

(d) Offline is deprecated

Q9. A feature-aware model was logged with `fs.log_model()` and a `batch_df` contains only the primary key `customer_id`. Which call generates predictions by automatically joining the required features?

(a) `model.predict(batch_df)`

(b) `fs.score_batch(model_uri, batch_df)`

(c) `mlflow.pyfunc.load_model(model_uri).predict(batch_df)`

(d) `batch_df.join(feature_table)`

Q10. Which namespace format is correct for a feature table in Unity Catalog?

(a) `catalog.schema.table`

(b) `database.table`

(c) `workspace.table`

(d) `hive_metastore.table`

Q11. Which MLflow API call manually logs a metric?

(a) `mlflow.log_metric()`

(b) `mlflow.log_param()`

(c) `mlflow.log_artifact()`

(d) `mlflow.set_tag()`

Q12. Which MLflow API call registers a model in Unity Catalog?

(a) `mlflow.log_model()`

(b) `mlflow.register_model()`

(c) `mlflow.save_model()`

(d) `mlflow.start_run()`

Q13. What replaced the old Staging and Production stages for model promotion in Unity Catalog?

(a) Tags only

(b) Aliases such as `@champion`

(c) Notebook revisions

(d) Git branches

Q14. How do you promote a challenger model to champion using aliases?

(a) Delete the old model version

(b) Move the `champion` alias to the new model version

(c) Rename the model

(d) Create a new experiment

Q15. What does `mlflow.search_runs()` return?

(a) A single run ID

(b) A pandas DataFrame of runs that can be sorted and filtered

(c) A trained model

(d) A list of experiments

Q16. A run ID is stored in `run_id` and the logged model name is `"model"`. Which line registers it as `"best_model"`?

(a) `mlflow.register_model(run_id, "best_model")`

(b) `mlflow.register_model(f"runs:/{run_id}/model", "best_model")`

(c) `mlflow.register_model(f"runs:/{run_id}/model")`

(d) `mlflow.register_model(f"runs:/{run_id}/best_model", "model")`

Q17. A data scientist wants one parent run for tuning and a child run per trial, starting runs manually. What should they do?

(a) Turn on Databricks Autologging

(b) Specify `nested=True` when starting each child run

(c) Start each child run inside the parent's code block without `nested`

(d) Give child runs the same experiment ID

Q18. Which UI element on the MLflow experiment page opens the notebook that created a run?

(a) The `MLmodel` artifact

(b) The `Models` link

(c) The `Source` link

(d) The `Start Time` link

Q19. Which function searches runs ordered by a metric ascending to find the lowest RMSE?

(a) `mlflow.search_runs(experiment_id, order_by=["metrics.rmse DESC"])`

(b) `mlflow.best_run(experiment_id, order_by=["metrics.rmse"])`

(c) `mlflow.search_runs(experiment_id, order_by=["metrics.rmse ASC"])`

(d) `mlflow.search_runs(experiment_id, filter="rmse < 1")`

Q20. On which page do you transition a registered model version from Staging to Production?

(a) The MLflow Model Registry home page

(b) The experiment page

(c) The model version page

(d) The model page only

Q21. Which MLflow API call logs an artifact?

(a) `mlflow.log_metric()`

(b) `mlflow.log_param()`

(c) `mlflow.log_artifact()`

(d) `mlflow.set_tag()`

Q22. What is the simplest Databricks-native way to make every training run reproducible months later?

(a) Save notebooks to a local drive

(b) Use MLflow tracking to log parameters, metrics, artifacts, and the model

(c) Print metrics to stdout

(d) Rely on cluster logs

Q23. What does Databricks AutoML produce besides a trained model?

(a) Only a model artifact

(b) A leaderboard, editable notebooks, and an automated EDA notebook

(c) A production endpoint

(d) A data warehouse

Q24. Which API runs classification AutoML?

(a) `databricks.automl.classify()`

(b) `databricks.automl.regress()`

(c) `databricks.automl.forecast()`

(d) `databricks.automl.train()`

Q25. AutoML is least appropriate when:

(a) You need a quick baseline

(b) You need a highly customized deep learning architecture

(c) You want automatic feature engineering

(d) You want to compare multiple algorithms

Q26. Which step must a data scientist perform outside an AutoML experiment?

(a) Model tuning

(b) Model evaluation

(c) Model deployment

(d) Exploratory data analysis

Q27. How do you begin using Databricks Runtime ML?

(a) Add an init script line
(b) Check a Databricks Runtime ML box
(c) Select a Runtime ML version from the Databricks Runtime Version dropdown
(d) Set `runtime-version="ml"` in the Spark session

Q28. Why is Databricks Runtime ML convenient for MLflow users?

(a) It removes the need for a cluster
(b) It preinstalls MLflow and other ML libraries
(c) It disables logging
(d) It only supports R

Q29. Which evaluation metric is not suitable for AutoML regression experiments?

(a) F1
(b) R-squared
(c) MAE
(d) MSE

Q30. Which file typically defines a model's format and dependencies in an MLflow run?

(a) `requirements.txt`
(b) `MLmodel`
(c) `README.md`
(d) `conda.yaml` only

Q31. What does `fs.log_model()` add to a model artifact?

(a) Hyperparameters
(b) Feature lookup metadata for automatic joins at inference
(c) A confusion matrix
(d) A serving endpoint

Q32. Which is a benefit of registering models in Unity Catalog over the workspace registry?

(a) Faster training
(b) Lineage, governance, and cross-workspace access
(c) Automatic hyperparameter tuning
(d) Free storage

Q33. What does an MLflow experiment group together?

(a) Runs of a related ML task

(b) Clusters

(c) Notebooks only

(d) Feature tables

Q34. Which API call starts an MLflow run?

(a) `mlflow.start_run()`

(b) `mlflow.log_run()`

(c) `mlflow.open_run()`

(d) `mlflow.new_run()`

Q35. Which is true about aliases in the Unity Catalog model registry?

(a) An alias can point to many versions at once

(b) An alias points to one model version and can be moved

(c) Aliases are immutable

(d) Aliases replace experiments

Q36. Which object stores the actual model files and metadata for a run?

(a) The artifact store

(b) The parameter log

(c) The metric log

(d) The experiment tag

Q37. Why should secrets not be logged as MLflow parameters?

(a) Parameters are stored in plain text and visible to experiment users

(b) Parameters are not persisted

(c) Parameters are encrypted by default

(d) Parameters are deleted after the run

2 B. ML Workflows

Q39. A Spark DataFrame `spark_df` must keep only rows where `price > 0`. Which code block works?

(a) `spark_df[spark_df["price"] > 0]`

(b) `spark_df.filter(col("price") > 0)`

(c) `SELECT * FROM spark_df WHERE price > 0`

(d) `spark_df.loc[spark_df["price"] > 0,:]`

Q40. Which line returns count, mean, standard deviation, min, max, and quantiles for numeric columns?

(a) `spark_df.summary()`

(b) `spark_df.stats()`

(c) `spark_df.describe().head()`

(d) `spark_df.printSchema()`

Q41. How do you remove outliers using IQR in Spark?

(a) Drop the top 10% of rows

(b) Filter values outside $[Q1 - 1.5 \cdot IQR,\ Q3 + 1.5 \cdot IQR]$

(c) Filter values outside $[Q1 - IQR,\ Q3 + IQR]$

(d) Replace all values with the mean

Q42. When is median imputation preferred over mean imputation?

(a) When data is symmetric

(b) When data is skewed or contains extreme outliers

(c) When there are no missing values

(d) Always

Q43. A data scientist replaces missing values with the median. A colleague says valuable information is lost. Which approach preserves the most information?

(a) Impute with the mean instead

(b) Let the algorithm handle missing values

(c) Drop all features with missing values

(d) Add a binary indicator feature marking which values were imputed

Q44. One-hot encoding is least appropriate for:

(a) Nominal categorical features

(b) Very high-cardinality categorical features

(c) Low-cardinality categorical features

(d) Tree-based models

Q45. A feature repository one-hot encodes all categorical variables. Why is this problematic?

(a) One-hot encoding is not supported by most libraries

(b) It depends on target values

(c) It is computationally intensive and should be done on small samples

(d) It is a potentially problematic strategy for some ML algorithms due to dimensionality and sparsity

Q46. How do you one-hot encode in Spark ML?

(a) `OneHotEncoder` alone

(b) `StringIndexer` then `OneHotEncoder`

(c) `VectorAssembler` then `StringIndexer`

(d) `Imputer` then `OneHotEncoder`

Q47. What does `VectorAssembler` do?

(a) Combines multiple columns into a single feature vector

(b) Splits a vector into columns

(c) Encodes strings

(d) Imputes missing values

Q48. A data scientist creates an `Imputer` and calls `transform` directly on it. Why does it fail?

(a) It does not impute both training and test sets

(b) `inputCols` and `outputCols` must be identical

(c) It should call `fit` instead of `transform`

(d) It does not fit the imputer to create an `ImputerModel`

Q49. When is log transformation appropriate?

(a) For right-skewed positive features

(b) For left-skewed negative features

(c) For categorical features

(d) For binary features

Q50. A pandas notebook must scale to big data with the least refactoring effort. What approach is recommended?

(a) Refactor to process data in parallel

(b) Refactor to the PySpark DataFrame API

(c) Refactor to the Scala Dataset API

(d) Refactor to Spark SQL

(e) Refactor to the pandas API on Spark

Q51. Which function applies a Python function to each group of a grouped DataFrame and returns a DataFrame?

(a) `applyInPandas`

(b) `groupedApplyInPandas`

(c) `mapInPandas`

(d) `predict`

Q52. A data scientist wants group-specific model inference in parallel. Which call fills the blank in `df.groupby("device_id").___(apply_model, schema=...)` ?

(a) `applyInPandas`

(b) `groupedApplyInPandas`

(c) `mapInPandas`

(d) `predict`

Q53. Which imputation strategy is most appropriate for a numeric feature with extreme outliers?

(a) Mean

(b) Median

(c) Mode

(d) Zero

Q54. What does a boxplot show?

(a) Only the mean

(b) Median, quartiles, and outliers

(c) Correlation

(d) Class balance

Q55. Which technique helps detect multicollinearity?

(a) Variance Inflation Factor

(b) Silhouette score

(c) ROC AUC

(d) Confusion matrix

Q56. Why is stratification important for classification splits?

(a) It preserves class proportions in each split

(b) It increases training speed

(c) It removes outliers

(d) It ensures reproducibility only

Q57. Which is a symptom of an imbalanced dataset?

(a) A 50/50 class split

(b) A 99% negative / 1% positive split

(c) A continuous target with normal distribution

(d) Missing values

3 C. Model Development

Q58. Which algorithm typically uses bagging?

(a) Gradient boosted trees
(b) K-means
(c) Random forest
(d) Linear regression

Q59. Spark ML linear regression on large data uses which approach for distributed training?

(a) Logistic regression
(b) Spark ML cannot distribute linear regression
(c) Iterative optimization such as L-BFGS
(d) Least-squares normal equations

Q60. Why can a Spark ML decision tree differ from a scikit-learn tree with identical data and hyperparameters?

(a) Spark ML tests every feature
(b) Spark ML automatically prunes overfit trees
(c) Spark ML tests more split candidates
(d) Spark ML tests a random sample of features
(e) Spark ML tests binned feature values as representative split candidates

Q61. What does Hyperopt's `fmin` do?

(a) Minimizes an objective function over a search space
(b) Maximizes accuracy only
(c) Trains a model
(d) Logs a metric

Q62. What does `SparkTrials` do?

(a) Distributes Hyperopt trials across a Spark cluster
(b) Runs trials on the driver only
(c) Saves models to the registry
(d) Encodes categorical features

Q63. What is the role of `tpe.suggest` in Hyperopt?

(a) Evaluates the objective function
(b) Specifies the MLflow experiment
(c) Suggests new hyperparameter configurations based on previous results
(d) Parallelizes the tuning process

Q64. A team has 8 workers and 5 hyperparameters. Which approach best leverages the cluster?

(a) Grid search on the driver

(b) Sequential manual tuning

(c) Hyperopt with `SparkTrials` and TPE

(d) Random search on the driver

Q65. With 3-fold CV and a grid of 3×2 hyperparameter values, how many models are trained in total?

(a) 6

(b) 9

(c) 18

(d) 36

Q66. With 3-fold CV and a grid of 3×2 values, how many models can be trained in parallel?

(a) 3

(b) 5

(c) 6

(d) 18

Q67. What is a primary advantage of Hyperopt over manual tuning or grid search?

(a) It efficiently explores the search space using algorithms like TPE

(b) Manual tuning is faster

(c) Grid search is more reliable

(d) Hyperopt needs more evaluations

Q68. When is `StringIndexer` used in a pipeline?

(a) To convert textual or categorical values to numeric indices

(b) To impute missing values

(c) To scale numeric features

(d) To assemble vectors

Q69. Why might a train-validation split be more efficient than k-fold CV?

(a) It requires testing fewer models, saving compute

(b) It reduces hyperparameter values to test

(c) It guarantees reproducibility

(d) It ensures bias-free training

Q70. When using a pipeline and a cross-validator, where should they be placed?

(a) Always put the cross-validator inside the pipeline

(b) If the pipeline includes estimators, put the entire pipeline inside the cross-validator

(c) If concerned about leakage, put the pipeline inside the cross-validator

(d) Always put the pipeline inside the cross-validator

Q71. A health organization wants to maximize the number of positive infection cases identified. Which metric should be used?

(a) RMSE
(b) Precision
(c) Area under the residual operating curve
(d) Accuracy
(e) Recall

Q72. When should F1 be used over accuracy?

(a) When classes are balanced
(b) When there are more than two classes
(c) When there is significant imbalance and avoiding false negatives is a priority
(d) When true positives and true negatives are equally important

Q73. A fraud dataset has a 1% positive class and a model reports 99% accuracy. What should you do?

(a) Deploy it
(b) Evaluate precision, recall, F1, and PR-AUC
(c) Increase epochs
(d) Add features

Q74. What does $R^2 = 0.85$ mean?

(a) 85% of predictions are correct
(b) The model explains 85% of the variance in the target
(c) The model has 85% precision
(d) The RMSE is 0.85

Q75. Why might RMSE be higher than MAE?

(a) RMSE penalizes large errors more heavily
(b) MAE squares errors
(c) RMSE ignores outliers
(d) MAE is in different units

Q76. Training accuracy is 99.5% and validation accuracy is 72%. What is the diagnosis and fix?

(a) Underfitting; increase depth
(b) Overfitting; reduce complexity or add regularization
(c) Good model; deploy
(d) Need more test data

Q77. What is the effect of increasing model complexity on bias and variance?

(a) Bias decreases, variance increases

(b) Bias increases, variance decreases

(c) Both decrease

(d) Both increase

Q78. A log(price) model is evaluated against actual price values and its RMSE is much larger than a price model. Which explanation is invalid?

(a) The second model is much more accurate

(b) They failed to exponentiate predictions before computing RMSE

(c) They failed to take the log of predictions in the first model

(d) The first model is much more accurate

(e) RMSE is invalid for regression

Q79. What is the 3-fold CV RMSE given fold RMSEs 10.0, 12.0, and 17.0?

(a) 13.0

(b) 17.0

(c) 12.0

(d) 39.0

Q80. What happens if the model is placed as a stage inside a pipeline and the cross-validator is the final stage?

(a) Training takes longer

(b) Feature engineering is computed using validation data, causing leakage

(c) CV is no longer reproducible

(d) The model is refit once more per fold

Q81. What does `CrossValidator` do in Spark ML?

(a) Performs k-fold CV with a `ParamGridBuilder`

(b) Trains one model

(c) Clusters data

(d) Imputes missing values

Q82. When is a train-validation split preferable to k-fold CV?

(a) When high interpretability is required

(b) When compute is limited, especially for large data or complex models

(c) When the algorithm is highly parallelizable

(d) When compute is unlimited

Q83. Which technique mitigates overfitting?

(a) Feature engineering and dimensionality reduction

(b) Increasing model complexity

(c) Using only one technique such as data augmentation

(d) All of the above

Q84. What is k-Nearest Neighbors' key hyperparameter tradeoff?

(a) Small k gives low bias and high variance; large k gives high bias and low variance

(b) Small k gives high bias; large k gives high variance

(c) k has no effect

(d) k only affects training time

Q85. Which kernel is commonly used for non-linear SVM classification?

(a) Linear

(b) RBF

(c) Identity

(d) Zero

Q86. What is gradient boosting?

(a) Trees trained in parallel on bootstrap samples

(b) Trees trained sequentially, each correcting previous errors

(c) A clustering method

(d) A dimensionality reduction method

Q87. What is the role of `max_depth` in tree models?

(a) Limits tree depth to control complexity and overfitting

(b) Sets the number of trees

(c) Sets the learning rate

(d) Sets the number of features

4 D. Model Deployment

Q89. What are the three model serving approaches tested on the exam?

(a) Batch, real-time, and streaming
(b) Batch, offline, and online
(c) Training, validation, and testing
(d) SQL, Python, and Scala

Q90. A scikit-learn model must score 100 million Delta records efficiently. What is the best approach?

(a) Load into pandas on the driver and loop
(b) Use `mlflow.pyfunc.spark_udf()` to distribute inference
(c) Export to CSV and score externally
(d) Create a real-time endpoint and call per record

Q91. How is streaming inference performed in Databricks?

(a) Using Delta Live Tables
(b) Using a SQL Warehouse
(c) Using the driver only
(d) Using a pandas UDF on a single node

Q92. How do you split traffic between real-time serving endpoints?

(a) Configure traffic splitting on the serving endpoint
(b) Duplicate the model
(c) Use a SQL query
(d) Rename the endpoint

Q93. What is the primary purpose of a real-time serving endpoint?

(a) Batch scoring
(b) Low-latency predictions accessible via REST API
(c) Training models
(d) Data storage

Q94. A team replaces one model with three new models of nearly equal latency. When is the new solution less efficient during inference?

(a) When if-else logic picks a model per prediction
(b) When average model latency exceeds the original model's size
(c) When fewer features are used
(d) When every record must be scored by all three models

Q95. What is a batch inference job in Databricks?

(a) Scoring a large dataset offline and writing predictions to a table

(b) A real-time REST endpoint

(c) A training job

(d) A cluster configuration

Q96. Which compute type is best for Spark SQL data prep plus Spark ML training in a notebook?

(a) Single Node cluster

(b) Standard (all-purpose) cluster

(c) SQL Warehouse

(d) None of these

Q97. Why is model monitoring important in production?

(a) Data distributions and relationships can drift over time

(b) Once deployed, models never degrade

(c) It replaces evaluation

(d) It improves training speed

Q98. What is data drift versus concept drift?

(a) Data drift changes input distributions; concept drift changes the feature-target relationship

(b) Both refer to the same thing

(c) Data drift changes labels; concept drift changes features

(d) Neither affects models

Q99. Which is a fairness concern when deploying ML?

(a) Biased training data leading to discriminatory outcomes

(b) Faster inference

(c) Lower storage cost

(d) Better dashboards

Q100. Which practice best guards against leakage in a production pipeline?

(a) Fit all transforms on the entire dataset

(b) Encapsulate preprocessing and model in a Pipeline fit only on training data

(c) Use the test set for feature selection

(d) Scale before splitting

5 Answer Key with Explanations

Q1. **B**. `get_table()` returns the table object; `.description` reads metadata. (A) creates a training set; (C) loads data; (D) returns the object without the description.

Q2. **B**. Account-level tables are governed and discoverable across workspaces. (A) and (D) are not the benefit; (C) is false.

Q3. **B**. `FeatureEngineeringClient` is the Unity Catalog Feature Store client. (A) is the legacy client; (C) and (D) are wrong.

Q4. **A**. `create_table()` creates. (B) writes; (C) builds a training set; (D) logs a model.

Q5. **B**. `write_table()` writes features. (A) creates; (C) scores; (D) retrieves.

Q6. **B**. Training sets use point-in-time lookups aligned to label timestamps. (A), (C), (D) are unrelated.

Q7. **B**. Leakage from post-label data causes training-serving skew. (A), (C), (D) do not match the symptom.

Q8. **B**. Online tables serve low-latency inference; offline tables are for batch. (A) reverses them; (C), (D) false.

Q9. **B**. `fs.score_batch()` uses logged feature metadata to join features automatically. (A), (C) assume features are already in the DataFrame; (D) is a plain join without automatic lookup.

Q10. **A**. Unity Catalog uses `catalog.schema.table`. (B), (C), (D) are not Unity Catalog formats.

Q11. **A**. `log_metric()` logs metrics. (B) logs params; (C) artifacts; (D) tags.

Q12. **B**. `register_model()` registers. (A) logs; (C) saves; (D) starts a run.

Q13. **B**. Aliases replace stages. (A), (C), (D) are not the replacement mechanism.

Q14. **B**. Promotion is moving the alias. (A) deletes history; (C), (D) do not promote.

Q15. **B**. It returns a pandas DataFrame of runs. (A) is too narrow; (C), (D) are false.

Q16. **B**. The URI must include `runs:/` and the artifact path. (A) passes only the run ID; (C) omits the registry name; (D) swaps artifact path and name.

Q17. **B**. Child runs must be started with `nested=True`. (A) does not nest; (C) raises an error; (D) only groups runs in the same experiment.

Q18. **C**. The Source link opens the notebook snapshot. (A) is a model metadata file; (B) links to registry; (D) opens run details.

Q19. **C**. `search_runs` with ascending order returns the lowest RMSE first. (A) orders descending; (B) is not the API; (D) filters rather than sorts.

Q20. **C**. Stage transitions happen on the model version page. (A), (B), (D) are not the direct page.

Q21. **C**. `log_artifact()` logs files. (A), (B), (D) log other things.

Q22. **B**. MLflow tracking logs params, metrics, artifacts, and models. (A), (C), (D) are not reproducible logs.

Q23. **B**. AutoML returns a leaderboard, notebooks, and an EDA notebook. (A) is incomplete; (C), (D) are not produced.

Q24. **A**. Classification AutoML uses `classify()`. (B) is regression; (C) forecasting; (D) not an API.

Q25. **B**. AutoML is less suited to highly custom deep learning. (A), (C), (D) are strengths.

Q26. **C**. Deployment is outside AutoML. (A), (B), (D) are automated.

Q27. **C**. Runtime ML is selected from the runtime dropdown. (A), (B), (D) do not select the runtime.

Q28. **B**. Runtime ML preinstalls MLflow and ML libraries. (A), (C), (D) are false.

Q29. **A**. F1 is for classification, not regression. (B), (C), (D) are regression metrics.

Q30. **B**. `MLmodel` defines format and dependencies. (A), (D) are dependency specs; (C) is documentation.

Q31. **B**. `log_model()` packages feature lookup metadata. (A), (C), (D) are not added by this call.

Q32. **B**. Unity Catalog adds lineage, governance, and cross-workspace access. (A), (C), (D) are false.

Q33. **A**. Experiments group related runs. (B), (C), (D) are not grouped by experiments.

Q34. **A**. `start_run()` starts a run. (B), (C), (D) are not APIs.

Q35. **B**. An alias points to one version and can be moved. (A) is false; (C) aliases are movable; (D) false.

Q36. **A**. Artifacts live in the artifact store. (B), (C), (D) store metadata, not files.

Q37. **A**. Parameters are stored in plain text and visible. (B), (C), (D) are false.

Q38. **B**. `filter(col(...))` is the PySpark way. (A) is pandas-like; (C) is SQL, not DataFrame code; (D) is pandas.

Q39. **A**. `summary()` returns count, mean, std, min, max, and quantiles. (B) is not an API; (C) lacks quantiles; (D) prints schema.

Q40. **B**. The 1.5×IQR rule is standard. (A), (C), (D) are wrong thresholds or methods.

Q41. **B**. Median is robust to skew and outliers. (A), (C), (D) are wrong contexts.

Q42. **D**. Indicator features preserve missingness information. (A), (B), (C) lose or ignore information.

Q43. **B**. High cardinality causes dimensionality explosion. (A), (C), (D) are not the main problem.

Q44. **D**. Premature one-hot can hurt some algorithms. (A), (B), (C) are false or irrelevant.

Q45. **B**. Spark ML uses StringIndexer then OneHotEncoder. (A), (C), (D) are wrong order or wrong tools.

Q46. **A**. VectorAssembler combines columns into a vector. (B), (C), (D) are different operations.

Q47. **D**. The imputer must be fit first. (A), (B), (C) are not the cause.

Q48. **A**. Log transform suits right-skewed positive data. (B), (C), (D) are wrong distributions.

Q49. **E**. pandas API on Spark minimizes refactoring. (A), (B), (C), (D) require larger rewrites.

Q50. **A**. `applyInPandas` applies per group and returns a DataFrame. (B) is not an API; (C) maps partitions; (D) is not an API.

Q51. **A**. Same as above. (B), (C), (D) are wrong.

Q52. **B**. Median is robust to outliers. (A), (C), (D) are not robust.

Q53. **B**. Boxplots show median, quartiles, and outliers. (A), (C), (D) are not boxplot content.

Q54. **A**. VIF detects multicollinearity. (B), (C), (D) measure other things.

Q55. **A**. Stratification preserves class proportions. (B), (C), (D) are unrelated.

Q56. **B**. 99/1 is severe imbalance. (A), (C), (D) are not imbalance symptoms.

Q57. **C**. Random forest uses bagging. (A) uses boosting; (B) is clustering; (D) is a single model.

Q58. **C**. Large data uses iterative optimizers such as L-BFGS. (A), (B), (D) are wrong.

Q59. **E**. Spark bins continuous features for distributed efficiency. (A), (B), (C), (D) are not the reason.

Q60. **A**. `fmin` minimizes the objective. (B), (C), (D) are wrong.

Q61. **A**. `SparkTrials` distributes trials. (B), (C), (D) are wrong.

Q62. **C**. `tpe.suggest` proposes new configurations. (A), (B), (D) are other responsibilities.

Q63. **C**. Hyperopt with `SparkTrials` and TPE leverages the cluster. (A), (B), (D) are inefficient.

Q64. **C**. $3 \times 2 \times 3 = 18$. (A), (B), (D) are wrong counts.

Q65. **C**. Six combinations can run in parallel. (A), (B), (D) are wrong counts.

Q66. **A**. Hyperopt explores more efficiently with TPE. (B), (C), (D) are false.

Q67. **A**. StringIndexer converts categories to indices. (B), (C), (D) are other transformers.

Q68. **B**. Train-validation trains fewer models. (A), (C), (D) are false.

Q69. **B**. Put the pipeline inside the CV when it contains estimators. (A), (C), (D) are not the general rule.

Q70. **E**. Recall maximizes identification of positives. (A), (B), (C), (D) measure other things.

Q71. **C**. F1 is for imbalance with false-negative priority. (A), (B), (D) are wrong conditions.

Q72. **B**. Evaluate precision, recall, F1, and PR-AUC. (A), (C), (D) are wrong actions.

Q73. **B**. R^2 is the fraction of variance explained. (A), (C), (D) are wrong interpretations.

Q74. **A**. RMSE penalizes large errors more. (B), (C), (D) are false.

Q75. **B**. Large train-validation gap means overfitting; reduce complexity. (A), (C), (D) are wrong.

Q76. **A**. Complexity lowers bias and raises variance. (B), (C), (D) are false.

Q77. **E**. RMSE is valid for regression. (A), (B), (C), (D) are possible explanations.

Q78. **A**. $(10 + 12 + 17)/3 = 13.0$. (B), (C), (D) are wrong.

Q79. **B**. Feature engineering on validation data leaks. (A), (C), (D) are not the consequence.

Q80. **A**. `CrossValidator` does k-fold CV with a parameter grid. (B), (C), (D) are other tools.

Q81. **B**. A split is cheaper than k-fold CV. (A), (C), (D) are false.

Q82. **A**. Feature engineering and dimensionality reduction mitigate overfitting. (B) worsens it; (C) is insufficient; (D) is false.

Q83. **A**. Small k means low bias and high variance. (B), (C), (D) are false.

Q84. **B**. RBF is common for non-linear SVM. (A) is linear; (C), (D) are not kernels.

Q85. **B**. Boosting trains sequentially. (A) is bagging; (C), (D) are other methods.

Q86. **A**. `max_depth` limits depth. (B), (C), (D) are other hyperparameters.

Q87. **A**. Batch, real-time, and streaming are the three serving approaches. (B), (C), (D) are not the exam's three.

Q88. **B**. `spark_udf` distributes inference. (A), (C), (D) do not scale.

Q89. **A**. Streaming inference uses Delta Live Tables. (B), (C), (D) are not streaming inference.

Q90. **A**. Traffic splitting is configured on the endpoint. (B), (C), (D) do not split traffic.

Q91. **B**. Real-time endpoints serve low-latency REST predictions. (A), (C), (D) are not the purpose.

Q92. **D**. Scoring every record with all three models multiplies cost. (A), (B), (C) do not necessarily reduce efficiency.

Q93. **A**. Batch inference scores offline and writes predictions. (B), (C), (D) are other concepts.

Q94. **B**. A Standard cluster supports SQL and Spark ML. (A) is single-node; (C) is SQL-only; (D) is false.

Q95. **A**. Drift degrades deployed models. (B), (C), (D) are false.

Q96. **A**. Data drift changes inputs; concept drift changes the relationship. (B), (C), (D) are false.

Q97. **A**. Biased data can cause discriminatory outcomes. (B), (C), (D) are not fairness concerns.

Q98. **B**. A Pipeline fit on training data prevents leakage. (A), (C), (D) cause leakage.